

C# / .NET Learning Roadmap

Topic Notes — Section 9: SOLID Principles and Core Design Patterns

Topics: 9.1 through 9.5

Date: 29 June 2026

Section 9 — SOLID Principles and Core Design Patterns

This section covers all five roadmap topics: 9.1 (SOLID Principles), 9.2 (Repository Pattern), 9.3 (Factory Pattern), 9.4 (Strategy Pattern), 9.5 (Manual Dependency Injection).

Teaching order: SOLID first (9.1) because every design pattern in 9.2-9.5 is a direct application of one or more SOLID principles.

9.1. SOLID Principles

Builds on: 2.1-2.9 (OOP -- classes, interfaces, inheritance, polymorphism), 2.9 (interfaces -- ISP and DIP depend on them heavily), 6.1 (delegates -- Func<T> sometimes used instead of interfaces for DIP).

What is SOLID and why does it exist?

SOLID is five design principles that produce code that is easy to change, test, understand, and extend. Violating them does not crash your program, but code that ignores them becomes increasingly painful to work with as it grows.

S -- Single Responsibility Principle (SRP)

The idea in one sentence: A class should have one reason to change.

Real-world analogy

A restaurant has a chef, a waiter, and a cashier -- each with one job. If one person does all three, everything breaks when that person is absent. Each role has one responsibility and one reason to change.

What it means in code

A class that does too much is called a God class. It becomes impossible to change without side effects.

```
// VIOLATES SRP -- one class does 5 unrelated things:
public class OrderService
{
    public async Task<Order> CreateOrderAsync(CreateOrderDto dto)
    {
        // 1. Validates
        if (dto.Amount <= 0)
            throw new ArgumentException("Amount must be positive");
        // 2. Saves to database
        var order = new Order(dto);
        await _context.Orders.AddAsync(order);
        await _context.SaveChangesAsync();
        // 3. Sends email
        await _smtpClient.SendAsync(BuildMessage(order));
        // 4. Logs analytics
        await _analyticsDb.InsertAsync(new OrderEvent { ... });
    }
}
```

```

        // 5. Generates PDF
        var pdf = PdfGenerator.Generate(order);
        await File.WriteAllBytesAsync(
            $"receipts/{order.Id}.pdf", pdf);
        return order;
    }
}

// FOLLOWS SRP -- each class has one job:
public class OrderValidator
    { public void Validate(CreateOrderDto dto) { ... } }
public class OrderRepository
    { public async Task<Order> SaveAsync(Order o) { ... } }
public class OrderEmailService
    { public async Task SendConfirmationAsync(Order o) { ... } }
public class OrderAnalyticsService
    { public async Task RecordCreatedAsync(Order o) { ... } }
public class OrderReceiptService
    { public async Task GenerateReceiptAsync(Order o) { ... } }

// Orchestrator -- just coordinates:
public class OrderService
{
    public async Task<Order> CreateOrderAsync(CreateOrderDto dto)
    {
        _validator.Validate(dto);
        var order = await _repo.SaveAsync(new Order(dto));
        await _email.SendConfirmationAsync(order);
        await _analytics.RecordCreatedAsync(order);
        await _receipt.GenerateReceiptAsync(order);
        return order;
    }
}

```

Simple to Advanced

Simple: Split a class that mixes data access and business logic.

```

// Simple fix:
public class EmailValidator
    { public bool IsValid(string email) { ... } }
public class UserRepository
    { public void Save(User user) { ... } }

```

Advanced -- CQRS: Each handler class handles exactly one command or query.

```

public class CreateOrderHandler
    : IRequestHandler<CreateOrderCommand, OrderDto>
{
    public async Task<OrderDto> Handle(
        CreateOrderCommand cmd, CancellationTokens ct)
        => /* only handles order creation */;
}

```

Must use / Must not use

Use SRP when:

- > Class name contains "And" (OrderAndEmailService)
- > Changing one feature requires testing unrelated features
- > Unit testing requires many unrelated mocks

Do NOT over-apply SRP:

- > Don't create a class for every single line of code
- > Goal is cohesion, not fragmentation

O -- Open/Closed Principle (OCP)

The idea in one sentence: A class should be open for extension but closed for modification.

Real-world analogy

A power strip: you plug in new devices without modifying the strip itself. The strip is open to new devices (extension) but you never rewire it (modification).

What it means in code

```
// VIOLATES OCP -- new discount requires editing this method:
public decimal CalculateDiscount(
    Order order, string discountType)
{
    if (discountType == "Percentage")
        return order.Amount * 0.10m;
    else if (discountType == "Fixed")
        return 50m;
    else if (discountType == "BOGO")      // added later
        return order.Amount / 2;
    else if (discountType == "Loyalty")  // added again
        return order.Amount * 0.15m;
    return 0;
}

// FOLLOWS OCP -- new types added by adding new classes:
public abstract class DiscountStrategy
{ public abstract decimal Calculate(Order order); }

public class PercentageDiscount : DiscountStrategy
{
    private readonly decimal _percent;
    public PercentageDiscount(decimal p) => _percent = p;
    public override decimal Calculate(Order o)
        => o.Amount * (_percent / 100);
}

public class BogoDiscount : DiscountStrategy
{ public override decimal Calculate(Order o) => o.Amount / 2; }

// Adding LoyaltyDiscount requires ZERO changes to existing code:
public class LoyaltyDiscount : DiscountStrategy
{
    private readonly int _years;
    public LoyaltyDiscount(int years) => _years = years;
```

```

    public override decimal Calculate(Order o)
        => o.Amount * (0.05m * _years);
}

// PricingService never changes when new discount types are added:
public class PricingService
{
    public decimal Apply(Order o, DiscountStrategy d)
        => o.Amount - d.Calculate(o);
}

```

Must use / Must not use

Use OCP when:

- > Same type of new thing added repeatedly each sprint
- > A switch/if-else chain grows with every release

Do NOT over-apply OCP:

- > Don't abstract on the first case (Rule of Three)
- > YAGNI -- You Aren't Gonna Need It

L -- Liskov Substitution Principle (LSP)

The idea in one sentence: Derived classes must be substitutable for their base class without breaking the program.

Real-world analogy

If someone asks for a vehicle, they expect it to move. A decorative toy car that doesn't move is technically 'a vehicle' but substituting it breaks the expectation.

Classic violation -- Rectangle/Square

```

public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
    public int Area() => Width * Height;
}

public class Square : Rectangle
{
    public override int Width
    {
        get => base.Width;
        set { base.Width = value; base.Height = value; }
    }
    public override int Height
    {
        get => base.Height;
        set { base.Height = value; base.Width = value; }
    }
}

```

```

void ResizeAndPrint(Rectangle r)
{
    r.Width = 5; r.Height = 10;
    Console.WriteLine(r.Area()); // expects 50
}
ResizeAndPrint(new Rectangle()); // 50 -- correct
ResizeAndPrint(new Square());    // 100 -- WRONG!

// FIX -- shared abstraction, no inheritance:
public interface IShape { int Area(); }

public class Rectangle : IShape
{
    public int Width { get; } public int Height { get; }
    public Rectangle(int w, int h) { Width = w; Height = h; }
    public int Area() => Width * Height;
}

public class Square : IShape
{
    public int Side { get; }
    public Square(int s) => Side = s;
    public int Area() => Side * Side;
}

```

Practical .NET example -- read-only collection violation

```

// VIOLATES LSP:
public class ReadOnlyOrderList : IList<Order>
{
    public void Add(Order item)
        => throw new NotSupportedException(); // LSP violated!
}

// FIX -- use correct interface matching actual capability:
public class ReadOnlyOrderList : IReadOnlyList<Order>
{ /* only read operations -- honest contract */ }

```

Must use / Must not use

Follow LSP when:

- > Derived class must honour ALL base class contracts
- > Implement every interface method meaningfully

LSP is VIOLATED when:

- > Derived class throws NotImplementedException
- > Overriding a method with an empty body to disable it
- > Returning null where base guarantees non-null

I -- Interface Segregation Principle (ISP)

The idea in one sentence: Clients should not be forced to depend on interfaces they do not use.

Real-world analogy

A Swiss Army knife: if someone only needs scissors, making them carry the entire knife with the saw, screwdriver, and bottle opener is wasteful. The interface should match what the client actually needs.

What it means in code

```
// VIOLATES ISP -- fat interface not all implementors need:
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);
    Task<List<Order>> GetAllAsync();
    Task<Order> CreateAsync(Order order);
    Task UpdateAsync(Order order);
    Task DeleteAsync(int id);
    Task<byte[]> ExportToCsvAsync(); // reporting only
    Task<byte[]> ExportToPdfAsync(); // reporting only
    Task BulkImportAsync( // admin only
        IEnumerable<Order> orders);
}

// Reporting repo forced to stub methods it doesn't need:
public class ReportingRepository : IOrderRepository
{
    public Task<Order?> GetByIdAsync(int id) { ... } // needed
    public Task<Order> CreateAsync(Order o) // not needed!
        => throw new NotSupportedException();
}

// FOLLOWS ISP -- small, focused interfaces:
public interface IOrderReader
{
    Task<Order?> GetByIdAsync(int id);
    Task<List<Order>> GetAllAsync();
}
public interface IOrderWriter
{
    Task<Order> CreateAsync(Order order);
    Task UpdateAsync(Order order);
    Task DeleteAsync(int id);
}
public interface IOrderExporter
{
    Task<byte[]> ExportToCsvAsync();
    Task<byte[]> ExportToPdfAsync();
}

// Full repository implements all:
public class SqlOrderRepository
    : IOrderReader, IOrderWriter, IOrderExporter { ... }

// Reporting service only depends on what it needs:
public class OrderReportingService
{
```

```

        private readonly IOrderReader _reader;
        public OrderReportingService(IOrderReader reader)
            => _reader = reader;
    }

```

Must use / Must not use

Use ISP when:

- > Interface grown with methods not all implementors use
- > Implementors throwing NotSupportedException on some methods
- > Tests require mocking methods never called in the test

Do NOT over-split interfaces:

- > Don't create one interface per method
- > If all consumers use all methods -- one interface is correct

D -- Dependency Inversion Principle (DIP)

The idea in one sentence: High-level modules should not depend on low-level modules. Both should depend on abstractions.

Real-world analogy

A light switch doesn't care which bulb is in the socket. Switch and bulb both depend on the standard socket interface. You can swap LED for incandescent without rewiring the switch.

What it means in code

```

// VIOLATES DIP -- hardwired to SQL implementation:
public class OrderService
{
    private readonly SqlOrderRepository _repo
        = new SqlOrderRepository("Server=prod-db;...");
    private readonly SmtEmailService _email
        = new SmtEmailService("smtp.zoho.com", 587);
    // Cannot test without real DB or SMTP server
}

// FOLLOWS DIP -- both sides depend on abstractions:
public interface IOrderRepository
{ Task<Order> SaveAsync(Order order); }
public interface IEmailService
{ Task SendConfirmationAsync(Order order); }

public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly IEmailService _email;

    public OrderService(
        IOrderRepository repo, IEmailService email)
    { _repo = repo; _email = email; }

    public async Task<Order> CreateAsync(CreateOrderDto dto)

```

```

    {
        var order = await _repo.SaveAsync(new Order(dto));
        await _email.SendConfirmationAsync(order);
        return order;
    }
}

// Swap freely -- no changes to OrderService needed:
public class SqlOrderRepository : IOrderRepository { ... }
public class PostgresOrderRepository : IOrderRepository { ... }
public class SmtEmailService : IEmailService { ... }
public class SendGridEmailService : IEmailService { ... }

// Testing -- inject fakes, no real DB needed:
var service = new OrderService(
    new InMemoryOrderRepository(),
    new FakeEmailService());

```

DIP in ASP.NET Core DI

```

// Program.cs -- map abstractions to implementations:
builder.Services
    .AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services
    .AddScoped<IEmailService, SmtEmailService>();

// Tests -- register test doubles instead:
services.AddScoped<IOrderRepository,
    InMemoryOrderRepository>();
services.AddScoped<IEmailService, FakeEmailService>();

```

SOLID working together

```

public class OrderProcessingService
{
    // DIP -- depends on abstractions:
    private readonly IOrderRepository _repo;
    private readonly IPaymentGateway _payment;
    private readonly IEmailService _email;

    public OrderProcessingService(
        IOrderRepository repo,    // DIP
        IPaymentGateway payment, // DIP
        IEmailService email)     // DIP
    { _repo = repo; _payment = payment; _email = email; }

    // SRP: only orchestrates order processing
    // OCP: new payment types need no edit here
    public async Task<ProcessResult> ProcessAsync(
        Order order, CancellationToken ct)
    {
        // LSP: IPaymentGateway guarantees a result:
        var result = await _payment.ProcessAsync(order.Amount, ct);
        if (!result.Success)
            return ProcessResult.Failed(result.Error);
    }
}

```

```

        order.MarkAsPaid(result.TransactionId);
        await _repo.SaveAsync(order, ct);
        await _email.SendReceiptAsync(order, ct);
        return ProcessResult.Success(order);
    }
}
// ISP: each interface focused -- IPaymentGateway has payment
//      methods only, IEmailService has email methods only

```

Key Takeaways for 9.1

Principle	Problem it solves	Tool
S - SRP	Classes that do too much	Split by concern
O - OCP	Switch growing every sprint	Polymorphism
L - LSP	Inheritance breaks callers	Honest interfaces
I - ISP	Fat, unused interfaces	Small focused interfaces
D - DIP	Hardwired, untestable code	Constructor injection

Memory aid: SOLID is CRISPS. Single-responsibility: one chip per bag. Open/Closed: add flavours without changing the factory. Liskov: every bag must contain crisps (not rocks). Interface Segregation: don't bundle crisps with cutlery no one needs. Dependency Inversion: the vending machine doesn't care which brand -- it just needs bags that fit the slot.

Debugging Tips (SOLID violations)

Symptom	Cause	Fix
Changing one feature breaks unrelated tests	SRP -- class does too much	Split by responsibility; one class per concern
Every new feature requires editing the same class	OCP -- not open for extension	Introduce polymorphism; each case becomes its own class
Substituting a derived class causes unexpected failures	LSP -- derived class breaks contract	Use narrower interface or fix the inheritance hierarchy
Mocking interface requires stubbing unused methods	ISP -- interface too fat	Split into smaller focused interfaces
Unit tests require a real database or email server	DIP -- hardwired dependencies	Inject abstractions through constructor; swap fakes in tests

9.2. Repository Pattern

Builds on: 9.1 (SOLID -- SRP, OCP, and DIP), 2.9 (interfaces), 7.3 (async/await -- all repository methods are async), 5.6 (LINQ with EF Core -- repositories encapsulate queries).

What is the Repository pattern and why does it exist?

Without a repository, business logic directly uses DbContext and writes EF Core LINQ queries everywhere. Testing becomes impossible without a real database. The Repository pattern introduces an abstraction layer between business logic and data access.

```
// The contract -- what business logic knows about:
```

```

public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(
        int id, CancellationToken ct = default);
    Task<List<Order>> GetByUserAsync(
        int userId, CancellationToken ct = default);
    Task<Order> CreateAsync(
        Order order, CancellationToken ct = default);
    Task UpdateAsync(
        Order order, CancellationToken ct = default);
    Task DeleteAsync(
        int id, CancellationToken ct = default);
    Task<bool> ExistsAsync(
        int id, CancellationToken ct = default);
}

// The implementation -- talks to EF Core:
public class SqlOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;
    public SqlOrderRepository(AppDbContext ctx) => _context = ctx;

    public async Task<Order?> GetByIdAsync(
        int id, CancellationToken ct = default)
        => await _context.Orders
            .AsNoTracking()
            .Include(o => o.Items)
            .FirstOrDefaultAsync(o => o.Id == id, ct);

    public async Task<List<Order>> GetByUserAsync(
        int userId, CancellationToken ct = default)
        => await _context.Orders
            .AsNoTracking()
            .Where(o => o.UserId == userId)
            .OrderByDescending(o => o.CreatedAt)
            .ToListAsync(ct);

    public async Task<Order> CreateAsync(
        Order order, CancellationToken ct = default)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync(ct);
        return order;
    }

    public async Task UpdateAsync(
        Order order, CancellationToken ct = default)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(ct);
    }

    public async Task DeleteAsync(
        int id, CancellationToken ct = default)
    {

```

```

        var order = await _context.Orders.FindAsync([id], ct);
        if (order is not null)
        {
            _context.Orders.Remove(order);
            await _context.SaveChangesAsync(ct);
        }
    }

    public async Task<bool> ExistsAsync(
        int id, CancellationToken ct = default)
        => await _context.Orders.AnyAsync(o => o.Id == id, ct);
}

```

Generic repository

```

public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(int id, CancellationToken ct = default);
    Task<List<T>> GetAllAsync(CancellationToken ct = default);
    Task<T> CreateAsync(T entity, CancellationToken ct = default);
    Task UpdateAsync(T entity, CancellationToken ct = default);
    Task DeleteAsync(int id, CancellationToken ct = default);
}

public class EfRepository<T> : IRepository<T> where T : class
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public EfRepository(AppDbContext context)
        => (_context, _dbSet) = (context, context.Set<T>());

    public virtual async Task<T?> GetByIdAsync(
        int id, CancellationToken ct = default)
        => await _dbSet.FindAsync([id], ct);

    public virtual async Task<List<T>> GetAllAsync(
        CancellationToken ct = default)
        => await _dbSet.AsNoTracking().ToListAsync(ct);

    public virtual async Task<T> CreateAsync(
        T entity, CancellationToken ct = default)
    {
        await _dbSet.AddAsync(entity, ct);
        await _context.SaveChangesAsync(ct);
        return entity;
    }

    public virtual async Task UpdateAsync(
        T entity, CancellationToken ct = default)
    {
        _dbSet.Update(entity);
        await _context.SaveChangesAsync(ct);
    }
}

```

```

    public virtual async Task DeleteAsync(
        int id, CancellationToken ct = default)
    {
        var entity = await GetByIdAsync(id, ct);
        if (entity is not null)
        {
            _dbSet.Remove(entity);
            await _context.SaveChangesAsync(ct);
        }
    }
}

// Specific interface extends generic with custom queries:
public interface IOrderRepository : IRepository<Order>
{
    Task<List<Order>> GetByUserAsync(
        int userId, CancellationToken ct = default);
    Task<decimal> GetTotalRevenueAsync(
        int userId, CancellationToken ct = default);
}

public class SqlOrderRepository
    : EfRepository<Order>, IOrderRepository
{
    public SqlOrderRepository(AppDbContext ctx) : base(ctx) { }

    public async Task<List<Order>> GetByUserAsync(
        int userId, CancellationToken ct = default)
    => await _dbSet
        .AsNoTracking()
        .Where(o => o.UserId == userId)
        .OrderByDescending(o => o.CreatedAt)
        .ToListAsync(ct);

    public async Task<decimal> GetTotalRevenueAsync(
        int userId, CancellationToken ct = default)
    => await _dbSet
        .Where(o => o.UserId == userId
            && o.Status == "Completed")
        .SumAsync(o => o.Amount, ct);
}

```

DI registration and in-memory repository for tests

```

// Program.cs:
builder.Services
    .AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped( // open generic
    typeof(IRepository<>), typeof(EfRepository<>));

// In-memory repository for unit tests:
public class InMemoryOrderRepository : IOrderRepository
{
    private readonly Dictionary<int, Order> _store = new();
    private int _nextId = 1;
}

```

```

public Task<Order?> GetByIdAsync(
    int id, CancellationToken ct = default)
    => Task.FromResult(
        _store.TryGetValue(id, out var o) ? o : null);

public Task<Order> CreateAsync(
    Order order, CancellationToken ct = default)
{
    order.Id = _nextId++;
    _store[order.Id] = order;
    return Task.FromResult(order);
}

public Task UpdateAsync(
    Order order, CancellationToken ct = default)
{ _store[order.Id] = order; return Task.CompletedTask; }

public Task DeleteAsync(
    int id, CancellationToken ct = default)
{ _store.Remove(id); return Task.CompletedTask; }

public Task<bool> ExistsAsync(
    int id, CancellationToken ct = default)
    => Task.FromResult(_store.ContainsKey(id));

public Task<List<Order>> GetAllAsync(
    CancellationToken ct = default)
    => Task.FromResult(_store.Values.ToList());

public Task<List<Order>> GetUserAsync(
    int userId, CancellationToken ct = default)
    => Task.FromResult(_store.Values
        .Where(o => o.UserId == userId).ToList());

public Task<decimal> GetTotalRevenueAsync(
    int userId, CancellationToken ct = default)
    => Task.FromResult(_store.Values
        .Where(o => o.UserId == userId
            && o.Status == "Completed")
        .Sum(o => o.Amount));
}

```

Must use / Must not use

Use Repository when:

- > Unit testing business logic without a real database
- > Multiple services query the same entity
- > Following Clean Architecture or DDD

Do NOT use Repository when:

- > Small script or prototype (YAGNI)
- > Every method is a one-line wrapper with no extra logic
(thin repository anti-pattern -- use DbContext directly)

Common mistakes:

- > Returning IQueryable<T> -- leaks EF Core outside
- > Putting business logic inside the repository
- > SaveChangesAsync() per method across a transaction

Key Takeaways for 9.2

- Repository = abstraction between business logic and data access
- Business logic depends on IOrderRepository -- never on DbContext directly
- Enables unit testing without a database via in-memory implementation
- Never return IQueryable<T> -- materialise inside the repository
- Generic IRepository<T> for CRUD; specific interface for entity queries

Memory aid: The repository is a librarian. You ask for orders by user -- you do not go rummage through the DbContext shelves yourself. In tests, replace the librarian with a simple in-memory catalogue.

Debugging Tips

Symptom	Cause	Fix
ObjectDisposedException when iterating result	DbContext disposed -- IQueryable returned from repo	Materialise with ToListAsync() inside the repo; return List<T>
Changes not saved after UpdateAsync	Entity not tracked -- AsNoTracking used on the fetch	Load without AsNoTracking for entities you will update
N+1 queries despite using repository	Missing Include in the repository method	Add appropriate Include/ThenInclude calls inside the repository
Duplicate SaveChanges saving partial state	Multiple repos each call SaveChangesAsync in a transaction	Use Unit of Work to control when SaveChangesAsync is called

9.3. Factory Pattern

Builds on: 9.1 (SOLID -- OCP and DIP), 9.2 (Repository), 2.2 (constructors -- factories replace complex constructor calls), 2.9 (interfaces -- factories return interface types), 6.1 (delegates -- Func<T> is a lightweight factory).

What is the Factory pattern and why does it exist?

A factory is an object or method whose sole responsibility is creating other objects. Instead of callers using new directly, they ask a factory to create the right object based on runtime data.

Real-world analogy: A car dealership. You say 'I need a 7-seater electric car.' The dealership figures out the right model. You don't go to the factory floor and assemble it yourself.

Variant 1 -- Static Factory Method

```
// Named creation -- intent is clear:
public class Money
{
    private decimal _amount;
    private string _currency;
```

```

private Money(decimal amount, string currency)
    => (_amount, _currency) =
        (amount, currency.ToUpperInvariant());

public static Money FromDollars(
    decimal dollars, string currency)
    => new Money(dollars, currency);

public static Money FromCents(
    long cents, string currency)
    => new Money(cents / 100m, currency);

public static Money Zero(string currency)
    => new Money(0m, currency);

public static Money? TryParse(string value)
{
    if (!decimal.TryParse(value[..^3], out var amount))
        return null;
    return new Money(amount, value[^3..]);
}

}

var price = Money.FromDollars(99.99m, "USD");
var zero = Money.Zero("INR");
var parsed = Money.TryParse("99.99USD");

```

Variant 2 -- Factory Class with interface

```

public interface IPaymentProcessorFactory
{
    IPaymentProcessor Create(string paymentMethod);
}

public class PaymentProcessorFactory
    : IPaymentProcessorFactory
{
    private readonly IConfiguration _config;
    private readonly IHttpClientFactory _http;

    public PaymentProcessorFactory(
        IConfiguration config, IHttpClientFactory http)
        => (_config, _http) = (config, http);

    public IPaymentProcessor Create(string paymentMethod)
        => paymentMethod.ToLowerInvariant() switch
        {
            "creditcard" => new StripePaymentProcessor(
                _config["Stripe:ApiKey"]!,
                _config["Stripe:Secret"]!),
            "banktransfer" => new BankTransferProcessor(
                _config["Bank:ApiUrl"]!,
                _http.CreateClient("BankApi")),
            "crypto" => new CryptoPaymentProcessor(
                _config["Crypto:WalletAddress"]!),
        }
}

```

```

        _ => throw new NotSupportedException(
            $"Method '{paymentMethod}' not supported")
    };
}

builder.Services
    .AddScoped<IPaymentProcessorFactory,
        PaymentProcessorFactory>();

```

Variant 3 -- Func<T> as a lightweight factory

```

builder.Services.AddScoped<
    Func<string, IPaymentProcessor>>(provider =>
        method => method switch
        {
            "creditcard" => provider
                .GetRequiredService<StripePaymentProcessor>(),
            "banktransfer" => provider
                .GetRequiredService<BankTransferProcessor>(),
            _ => throw new NotSupportedException(
                $"Unknown: {method}")
        });

// Usage:
public class CheckoutService
{
    private readonly Func<string, IPaymentProcessor> _factory;

    public CheckoutService(
        Func<string, IPaymentProcessor> factory)
        => _factory = factory;

    public async Task CheckoutAsync(
        Order order, string paymentMethod)
    {
        var processor = _factory(paymentMethod);
        await processor.ProcessAsync(order.Amount);
    }
}

```

Factory using ActivatorUtilities for DI-aware creation

```

public class ReportGeneratorFactory
{
    private readonly IServiceProvider _provider;

    public ReportGeneratorFactory(IServiceProvider p)
        => _provider = p;

    public IReportGenerator Create(ReportType type)
        => type switch
        {
            ReportType.Sales =>
                ActivatorUtilities
                    .CreateInstance<SalesReportGenerator>()

```

```

        _provider),
        ReportType.Inventory =>
            ActivatorUtilities
                .CreateInstance<InventoryReportGenerator>(
                    _provider),
        _ => throw new NotSupportedException()
    };
}

```

Must use / Must not use

Use Factory when:

- > Object creation requires complex logic callers should not know
- > Different implementations needed based on runtime input
- > Named construction is clearer than overloaded constructors

Do NOT use Factory when:

- > Object is simple -- `new Order()` needs no factory
- > DI can handle creation directly -- register and inject
- > One type that will never have alternatives

Key Takeaways for 9.3

- Static factory method -- named construction; replaces confusing overloads
- Factory class -- dedicated class; centralises runtime creation decisions
- `Func<T>` -- lightweight delegate factory for simple cases
- Factories return interface types -- callers never see the concrete type
- Use `ActivatorUtilities.CreateInstance<T>` for DI-aware creation

Memory aid: A factory is a vending machine. You press D3 (the key = payment method, report type) and the machine gives you the right item. You do not see the mechanism inside.

Debugging Tips

Symptom	Cause	Fix
NotSupportedException from factory at runtime	Input key not in the switch/dictionary	Add the missing case; validate input before passing to factory
Factory leaks concrete type through interface	Factory method return type is the concrete class	Change return type to interface: <code>IPaymentProcessor</code> not <code>StripeProcessor</code>
Factory objects have missing dependencies	Manually newing up objects instead of resolving from DI	Use <code>_provider.GetRequiredService<T>()</code> or <code>ActivatorUtilities</code>
Singleton factory creates Scoped services incorrectly	Captive dependency -- Singleton holds Scoped services	Register factory as Scoped or use <code>IServiceScopeFactory</code>

9.4. Strategy Pattern

Builds on: 9.1 (SOLID -- canonical implementation of OCP and DIP), 9.3 (Factory -- factories often create strategies), 2.7 (polymorphism), 6.1 (`Func<T>` is a lightweight strategy).

What is the Strategy pattern and why does it exist?

A strategy is a swappable algorithm or behaviour. Instead of hardcoding one way to do something, you define an interface for the behaviour and inject whichever implementation the situation calls for.

Real-world analogy: Navigation apps give you strategy choices -- fastest route, shortest route, avoid tolls. Adding scenic route does not require rebuilding the GPS -- it is a new strategy plugged into the same framework.

The strategy pattern -- core structure

```
// Step 1 -- Define the strategy interface:
public interface IPricingStrategy
{
    decimal Calculate(Order order);
    string Name { get; }
}

// Step 2 -- Concrete strategies:
public class RegularPricingStrategy : IPricingStrategy
{
    public string Name => "Regular";
    public decimal Calculate(Order o) => o.Amount;
}

public class PremiumPricingStrategy : IPricingStrategy
{
    public string Name => "Premium";
    public decimal Calculate(Order o) => o.Amount * 0.90m;
}

public class VipPricingStrategy : IPricingStrategy
{
    public string Name => "VIP";
    public decimal Calculate(Order o) => o.Amount * 0.80m;
}

// Adding a new strategy requires ZERO changes to existing code:
public class LoyaltyPricingStrategy : IPricingStrategy
{
    private readonly int _years;
    public string Name => "Loyalty";
    public LoyaltyPricingStrategy(int years) => _years = years;
    public decimal Calculate(Order o)
        => o.Amount * (1 - 0.05m * _years);
}

// Step 3 -- Context uses the strategy:
public class PricingService
{
    private readonly IPricingStrategy _strategy;

    public PricingService(IPricingStrategy strategy)
        => _strategy = strategy;
}
```

```

        public decimal CalculatePrice(Order order)
            => Math.Max(_strategy.Calculate(order), 0);
    }

```

Runtime strategy selection

```

// Strategy selector:
public class PricingStrategySelector
{
    private readonly IReadOnlyDictionary<string, IPricingStrategy>
        _strategies;

    // IEnumerable<IPricingStrategy> collects ALL registered:
    public PricingStrategySelector(
        IEnumerable<IPricingStrategy> strategies)
        => _strategies = strategies.ToDictionary(
            s => s.Name,
            StringComparer.OrdinalIgnoreCase);

    public IPricingStrategy Select(string customerType)
        => _strategies.TryGetValue(
            customerType, out var s)
            ? s : _strategies["Regular"];
}

// DI registration -- register ALL strategies:
builder.Services
    .AddScoped<IPricingStrategy, RegularPricingStrategy>();
builder.Services
    .AddScoped<IPricingStrategy, PremiumPricingStrategy>();
builder.Services
    .AddScoped<IPricingStrategy, VipPricingStrategy>();
builder.Services.AddScoped<PricingStrategySelector>();
// ASP.NET Core injects IEnumerable<IPricingStrategy> with all

```

Composite strategy -- validation pipeline

```

public interface IOrderValidator
{
    ValidationResult Validate(Order order);
}

public class AmountValidator : IOrderValidator
{
    public ValidationResult Validate(Order order)
    {
        if (order.Amount <= 0)
            return ValidationResult.Fail(
                "Amount must be positive");
        if (order.Amount > 1_000_000)
            return ValidationResult.Fail(
                "Amount exceeds maximum");
        return ValidationResult.Ok();
    }
}

// Composite -- runs all strategies, collects results:

```

```

public class CompositeOrderValidator
{
    private readonly IEnumerable<IOrderValidator> _validators;

    public CompositeOrderValidator(
        IEnumerable<IOrderValidator> validators)
        => _validators = validators;

    public IReadOnlyList<string> ValidateAll(Order order)
        => _validators
            .Select(v => v.Validate(order))
            .Where(r => !r.IsValid)
            .Select(r => r.ErrorMessage!)
            .ToList()
            .AsReadOnly();

    public bool IsValid(Order order)
        => !ValidateAll(order).Any();
}

builder.Services.AddScoped<IOrderValidator, AmountValidator>();
builder.Services
    .AddScoped<IOrderValidator, CustomerValidator>();
builder.Services.AddScoped<IOrderValidator, StockValidator>();
builder.Services.AddScoped<CompositeOrderValidator>();

```

Must use / Must not use

Use Strategy when:

- > Multiple algorithms for the same operation exist
- > Algorithm changes based on runtime data
- > if/else or switch grows every sprint
- > Each algorithm should be tested independently

Do NOT use Strategy when:

- > Only one algorithm exists and no second is anticipated
- > A simple Func<T> would be clearer

Key Takeaways for 9.4

- Strategy = swappable algorithm defined by an interface and injected
- Eliminates growing if/else -- each case becomes its own class
- IEnumerable<IStrategy> collects all registered strategies automatically
- Selector/dictionary picks the right strategy at runtime (O(1) lookup)
- Func<T> is a lightweight strategy for simple stateless one-liners
- Composite strategy runs all strategies and aggregates results

Approach	When to use
Strategy class + interface	Complex, stateful, testable algo
Func<T> delegate	Simple stateless one-liner
IEnumerable<IStrategy>+selector	Multiple, runtime selection
Composite strategy	All run, results aggregated

Memory aid: Strategy is a GPS mode selector. The destination is fixed. The route algorithm varies -- fastest, shortest, avoid tolls. Adding scenic route is a new mode plugged in -- no GPS rebuild required.

Debugging Tips

Symptom	Cause	Fix
IEnumerable<IStrategy> is empty at runtime	Strategies not registered in DI	Add AddScoped<IStrategy, ConcreteStrategy>() for each
Wrong strategy selected at runtime	Key mismatch between selector and strategy Name	Use StringComparer.OrdinalIgnoreCase; log the selected key
ToDictionary throws on duplicate Names	Two strategies have the same Name value	Use GroupBy + Last(): strategies.GroupBy(s => s.Name) .ToDictionary(g => g.Key, g => g.Last(), ...)
Strategy has NullReferenceException	Strategy dependencies not resolved by DI	Ensure all strategy constructor dependencies are registered

9.5. Manual Dependency Injection (Concept, Constructor Injection)

Builds on: 9.1 (SOLID -- DIP is the principle; DI is the mechanism), 9.2-9.4 (all rely on constructor injection), 2.2 (constructors), 2.9 (interfaces).

What is Dependency Injection and why does it exist?

DI is the practice of providing an object's dependencies from the outside rather than letting the object create them itself. Manual DI means understanding DI as a concept first -- before relying on a container. Once you understand manual DI, the ASP.NET Core container becomes obvious: it is just an automated version of what you'd do by hand.

Real-world analogy: A chef who receives ingredients from a supplier is decoupled -- swap the supplier, the technique stays the same. The restaurant (DI container) decides which supplier delivers what. The chef (your class) just declares what they need.

Constructor injection -- the primary DI pattern

```
// WITHOUT DI -- class creates its own dependencies:
public class OrderService
{
    private readonly SqlOrderRepository _repo
        = new SqlOrderRepository("Server=prod-db;...");
    private readonly SmtEmailService _email
        = new SmtEmailService("smtp.zoho.com", 587);
    // Cannot test without real DB or SMTP server
}

// WITH Constructor Injection:
```

```

public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly IEmailService _email;

    public OrderService(
        IOrderRepository repo, IEmailService email)
    {
        _repo = repo ?? throw new ArgumentNullException(nameof(repo));
        _email = email ?? throw new ArgumentNullException(nameof(email));
    }

    public async Task<Order> CreateAsync(CreateOrderDto dto)
    {
        var order = await _repo.SaveAsync(new Order(dto));
        await _email.SendConfirmationAsync(order);
        return order;
    }
}

```

Manual DI -- wiring everything by hand

```

// What DI containers automate:
var dbContext = new AppDbContext(connectionString);

IOrderRepository orderRepo =
    new SqlOrderRepository(dbContext);
IEmailService emailService =
    new SmtpEmailService("smtp.zoho.com", 587);
IReceiptService receiptSvc =
    new PdfReceiptService("/var/receipts");

var orderService =
    new OrderService(orderRepo, emailService, receiptSvc);
var controller =
    new OrdersController(orderService);

// This IS DI -- done manually.
// Swap SmtpEmailService for SendGridEmailService: one line.
// Tests swap SqlOrderRepository for InMemory: one line.

```

The three DI patterns

Constructor injection -- the standard and recommended pattern:

```

public class OrderService
{
    private readonly IOrderRepository _repo;
    // Required; validated at construction time:
    public OrderService(IOrderRepository repo)
        => _repo = repo ??
            throw new ArgumentNullException(nameof(repo));
}

```

Property injection -- for optional dependencies:

```

public class OrderService
{
    public ILogger<OrderService>? Logger { get; set; }
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;
}
// Use sparingly -- class may be used without property set

```

Method injection -- dependency varies per call:

```

public async Task<string> GenerateAsync(
    IReportFormatter formatter)
{
    var orders = await _repo.GetAllAsync();
    return formatter.Format(orders);
}
// Use when dependency changes per call, not per instance

```

Service lifetimes

```

// Transient -- new instance every injection:
builder.Services
    .AddTransient<IEmailService, SmtEmailService>();
// Use for: lightweight, stateless services

// Scoped -- one per HTTP request (most common):
builder.Services
    .AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped<AppDbContext>();
// Use for: DbContext, request-consistent state

// Singleton -- one for entire application lifetime:
builder.Services
    .AddSingleton<IExchangeRateCache,
        InMemoryExchangeRateCache>();
// Use for: caches, config, thread-safe services

// CAPTIVE DEPENDENCY -- BUG:
builder.Services
    .AddSingleton<IOrderService, OrderService>();
builder.Services
    .AddScoped<IOrderRepository, SqlOrderRepository>();
// Singleton captures Scoped DbContext -- never released!

// Safe dependency directions:
// Singleton -> Singleton only
// Scoped    -> Singleton + Scoped
// Transient -> Singleton + Scoped + Transient

```

DI violations -- what to look for

```

// VIOLATION 1 -- new inside a class for a dependency:
private readonly IOrderRepository _repo
    = new SqlOrderRepository(); // hardwired!
// Fix: inject IOrderRepository through constructor

// VIOLATION 2 -- static dependencies:

```

```

await EmailSender.SendAsync(dto.Email); // static call!
// Fix: inject IEmailService through constructor

// VIOLATION 3 -- service locator (anti-pattern):
var repo = _provider
    .GetRequiredService<IOrderRepository>();
// Fix: inject IOrderRepository directly
// Exception: Factory classes may legitimately use IServiceProvider

```

Manual DI in unit tests

```

// Build any object graph without a container:
public class OrderServiceTests
{
    private OrderService BuildService(
        IOrderRepository? repo = null,
        IEmailService? email = null)
    {
        repo ??= new InMemoryOrderRepository();
        email ??= new FakeEmailService();
        return new OrderService(
            repo, new OrderValidator(), email);
    }

    [Fact]
    public async Task CreateAsync_SendsConfirmation()
    {
        var fakeEmail = new FakeEmailService();
        var service = BuildService(email: fakeEmail);

        await service.CreateAsync(new CreateOrderDto
        {
            CustomerName = "Sathish",
            CustomerEmail = "s@zoho.com",
            Amount = 500m
        }, CancellationToken.None);

        Assert.Single(fakeEmail.SentEmails);
        Assert.Equal("s@zoho.com",
            fakeEmail.SentEmails[0].To);
    }
}

public class FakeEmailService : IEmailService
{
    public List<(string To, string Subject)>
        SentEmails { get; } = new();

    public Task SendConfirmationAsync(
        Order order, CancellationToken ct)
    {
        SentEmails.Add(
            (order.CustomerEmail, "Order Confirmation"));
        return Task.CompletedTask;
    }
}

```

}

How all of Section 9 connects

SOLID (9.1) provides the WHY:

- SRP -> Each class has one job
- OCF -> Add behaviour by adding, not modifying
- DIP -> Depend on abstractions, not concretions

Repository (9.2) applies DIP + SRP:

Business logic -> IOrderRepository, not SqlOrderRepository

Factory (9.3) applies OCP + DIP:

New types added without modifying factory callers

Strategy (9.4) applies OCP + DIP:

New algorithms added without modifying the context class

Constructor Injection (9.5) is HOW DIP is implemented:

- Dependencies flow in through constructors
- Program.cs wires everything together
- Tests wire their own fakes without touching production code

Key Takeaways for 9.5

- DI = providing dependencies from outside rather than creating them inside
- Constructor injection -- the standard: declare in constructor, receive from caller
- Manual DI = wiring the object graph by hand -- what containers automate
- Transient (new per injection), Scoped (one per request), Singleton (one forever)
- Scoped can depend on Singletons but not vice versa -- captive dependency is a bug
- Service locator (IServiceProvider in business logic) is an anti-pattern

Lifetime	Instance	Use for
-----+	-----+	-----
Transient	New every injection	Lightweight, stateless helpers
Scoped	One per HTTP request	DbContext, request-scoped state
Singleton	One per app lifetime	Caches, config, thread-safe

Memory aid: Constructor injection is like a job application asking for qualifications upfront. The class declares 'I need IOrderRepository, IEmailService.' The container (HR) finds the right candidates and makes the introductions. The class never recruits its own staff.

Debugging Tips

Symptom	Cause	Fix
Cannot consume scoped service from singleton	Captive dependency -- Singleton holds a Scoped service	Change Singleton to Scoped, or use IServiceScopeFactory for a new scope
No service for type Ixxx	Interface not registered in DI	Add builder.Services.AddScoped<Ixxx, ConcreteXxx>() in Program.cs
Wrong implementation injected	Multiple registrations for same interface -- last one wins	Check registration order; use factory if multiple implementations needed

Symptom	Cause	Fix
Stale DbContext data across requests	Singleton or static DbContext	Register as Scoped: <code>builder.Services.AddDbContext<AppDbContext>(...)</code>
Test requires a real database	Business logic depends on DbContext directly	Inject <code>IOrderRepository</code> (interface) -- swap to in-memory fake in tests

Section 9 complete. All 5 roadmap topics (9.1-9.5) covered: SOLID Principles, Repository Pattern, Factory Pattern, Strategy Pattern, and Manual Dependency Injection. Say next to begin Section 10: Unit Testing and Debugging.